Two clocks, one cut: may–can concordance and revocation in MLS groups

Daniel Sobral[1]

[1]*Quside Technologies, Barcelona, Spain*

(Dated: 25 October 2026)

An authorization service decides who *may* read or act in a group. A group-key protocol such as MLS decides who *can*. The two descriptions run on different clocks, and a party removed from the first may keep the keys of the second. We give a minimal model in which both evolve, and we fix the event that makes the two clocks comparable: the *cut*, the ledger revision at which an action is released. A protocol is in *may–can concordance* when whoever can exercise a capability on an action may do so at that action's cut. A no-go result shows that oblivious composition of a correct authorization service with a correct group protocol is generically discordant. We give a construction made of four local rules (a fence, a removal commit, a release rule and an acceptance rule). We prove that it is concordant for reading and for acting, including delegated and invoked authority with cascading revocation. Read concordance factorizes into two parts. The first is a combinatorial statement about rosters, proved here by induction over the protocol's transitions. The second is a cryptographic property of MLS, key confinement: an epoch key is hidden from every party outside that epoch's roster. We derive key confinement from published theorems. The symbolic form is a corollary of the machine-checked analysis of TreeKEM. The computational form follows from the ETK theorem for RFC 9420, because every commit the construction issues is an operation ETK models. Write concordance reduces to signature unforgeability. We also searched every reachable state of a finite model, 1.56×10^6 states, against an adversary that never forgets a key. The search finds no violation, and deleting any one of nine rules or conditions produces a counterexample that the search finds unaided. A second no-go result states the price: the authorization ledger must be reachable at every release.

I. INTRODUCTION

Consider a group with three members, Alice, Bob and Carol, who share an end-to-end encrypted channel. An external authority, the *ledger*, records who may read. At some instant the ledger revokes Alice. The ledger is correct: every later query returns "Alice may not read." The group protocol is also correct. Alice is still in the current epoch and holds its key, because nobody has yet committed her removal. If Bob now sends a fresh message in the old epoch, Alice reads it. Neither component failed. Their composition did.

The difficulty is not a bug in either component but a mismatch of descriptions. The ledger describes authorization as a function of its revision number v. The group describes capability as a function of its epoch e. Nothing ties the two clocks together, so the phrase "after Alice was revoked" has no meaning for Bob's message until we say *which event* defines "now" for that message. The situation resembles simultaneity in relativity: before one can compare two clocks, one must fix the event that serves as a common reference. Here that event is the moment the ledger *releases* the action. We call the ledger revision at that moment the action's *cut*.

With the cut fixed, the requirement is easy to state. Every party that *can* exercise a capability on an action must have been one that *may*, at the action's cut. When the two descriptions agree in this sense we say they are in *may–can concordance*, or simply that the protocol is *concordant*. When they disagree it is *discordant*. Figure 1 shows the *discordance window* that opens when the release ignores the roster.

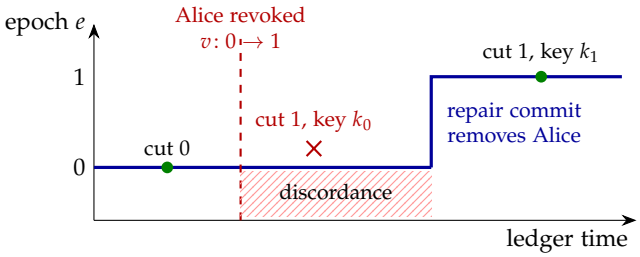


FIG. 1. Two clocks. Horizontal: the ledger's history. Vertical: the group's epoch. Alice is revoked at revision 1, but she leaves the group only at the next epoch. A message released inside the hatched discordance window has cut 1 (Alice may not read) yet is protected under k_0, which Alice holds (she can). Messages released before the revocation (left dot) or after the repair (right dot) are concordant. The construction of Sec. IV closes a *fence* over the window, so no release happens there.

Results. We work in a model that keeps only what the argument needs: a ledger of facts with lifetimes, justifications built from facts, and a group whose epochs have rosters and keys (Sec. II). In this model:

1. Concordance is defined for reading and for acting [Eqs. (5) and (6)]. A no-go result shows that oblivious composition violates it (Proposition 1).

2. Four local rules suffice for concordance (Sec. IV). The proof of read concordance *factorizes* into two parts. The first is a combinatorial lemma about

rosters, roster inclusion (Lemma 2), proved by induction. The second is one property of MLS, key confinement: an epoch key is hidden from every party outside that epoch's roster. The factorization theorem (Theorem 1) composes the two. Write concordance needs only signature unforgeability (Theorem 2).

3. Key confinement follows from published results under five conditions K1–K5, all enforced by the construction (Sec. V B). Symbolically, it is a corollary of the machine-checked TreeKEM theorem (Proposition 2). Computationally, it follows from the ETK theorem (Proposition 3). To make that possible, the policy binding lives in the ledger's signed acknowledgement of each commit, not in the MLS group context.
4. Delegated authority and invocations need no new theory. A delegation chain is a justification with more facts, and cascading revocation follows from one locality property (Sec. V E).
5. The cost is a ledger contact per release, and a second no-go result shows that it cannot be avoided at the release cut (Proposition 5). Under stated assumptions the construction makes progress after a revocation (Proposition 4).
6. An exhaustive finite-state search supports the claims (Sec. VI). Its adversary never forgets a key, and every rule is shown necessary by an automatically found counterexample.

Key confinement is the only cryptographic property of MLS that we use. Appendix A matches it, case by case, against the exact statements of the two analyses it rests on.

II. MODEL

The ledger. The ledger is a totally ordered sequence of revisions $v = 0, 1, 2, \ldots$. Its content is a set of *facts* F. A fact is a root grant, a delegation from one party to another, or an invocation that lets one party act for another. At revision v a subset $L_v \subseteq F$ of facts is *live*. A revocation removes a fact from the live set. Re-granting never revives a fact; it issues a new one with a new generation number. The lifetime of every fact is therefore an interval:

$$f \in L_u,\ f \in L_w,\ u < v < w \implies f \in L_v. \tag{1}$$

Justifications. An *action type* $\alpha = (p, r, x)$ names a party p, a right $r \in \{\text{read}, \text{write}, \text{admin}, \text{admit}\}$ and a resource or group x. A *justification* is a finite set of facts $J \subseteq F$. We write $J \Vdash_v \alpha$ when J justifies α at revision v. The single structural property we require is that authorization depends on time only through liveness:

$$J \Vdash_v \alpha \iff J \subseteq L_v \ \wedge\ \text{Scope}(J, \alpha), \tag{2}$$

where the predicate Scope does not depend on v. It checks everything about the *content* of the facts: that a delegation chain is connected, that each link narrows the rights and resources of its parent, and that the chain ends at p with right r on x. Equation (2) says that every change of authorization is a ledger event. A delegation that expires at a wall-clock time must be modeled as a revocation recorded on the ledger at that time.

The *readers* of group g at revision v are

$$\text{Rd}_v(g) = \{\, p : \exists J \ \ J \Vdash_v (p, \text{read}, g) \,\}. \tag{3}$$

The group. The group moves through epochs $e = 0, 1, 2, \ldots$. Epoch e has a roster M_e of members (one entry per device incarnation) and an epoch key k_e, from which the message keys of that epoch are derived. A party *holds* everything it has ever received. No party is ever assumed to forget. We write $S \vdash k$ when the parties in set S, pooling everything they have ever held, can compute k.

Actions and their cut. A *protected action* a is a single read-protected message, a write, or a control operation (a proposal or commit, which is an admin or admit action). Every action passes through one atomic *release* step at the ledger. The ledger revision read by that step is the action's cut,

$$v(a) = \text{revision at the release of } a. \tag{4}$$

A read action is protected under the key of an epoch $e(a)$. Neither encryption, transmission nor receipt moves the cut. A message released before a revocation and read afterwards is judged at the earlier revision. This is a deliberate choice of semantics: the *release cut*. Causal or lease-based cuts are possible and lead to different properties, which we do not treat.

III. CONCORDANCE

Definition 1 (Read concordance)**.** *A protocol is* read-concordant *if, in every execution, for every released read action a in group g,*

$$\{p\} \vdash k_{e(a)} \implies p \in \text{Rd}_{v(a)}(g). \tag{5}$$

In words, nobody can derive the key of a message without having been a reader at that message's cut. Equation (5) is the symbolic statement. The computational statement is an indistinguishability game rather than an event. Fix a challenge action a in group g. Let the *coalition* $C_a = \{\, p : p \notin \text{Rd}_{v(a)}(g) \,\}$ be all parties that were not readers at its cut. The adversary controls the network, receives everything the members of C_a ever hold, at any time, and chooses two equal-length messages m_0, m_1. The honest sender protects m_b for a uniform bit b. The protocol is read-concordant if no efficient adversary guesses b with probability non-negligibly above $1/2$. We denote that excess by Adv^{read}.

Two things lie outside this definition, deliberately. A party that *was* a reader at the cut may read the message at any later time and leak it; no protocol can prevent that. Plaintext obtained before a revocation is not erased.

Definition 2 (Write concordance)**.** *A protocol is* write-concordant *if, whenever an honest receiver accepts an action a of type α,*

$$\exists J:\ J \Vdash_{v(a)} \alpha, \quad \textit{bytes accepted} = \textit{bytes released}, \tag{6}$$

and each receiver accepts each release at most once.

Write concordance concerns events observed at honest receivers, so it is a property of executions directly. It applies equally to writes, to administrative commits (where the "receiver" is the sequencer and every member processing the commit) and to admissions.

Proposition 1 (No-go for oblivious composition)**.** *Let the ledger be an ideal authorization service and the group an ideal group-key protocol, each meeting its own specification. Call the composition* oblivious *if the release of an action does not depend on the group's roster. Every oblivious composition has an execution violating Eq. (5).*

Proof. Start with $M_0 = \{A, B\}$, both readers at revision 0. The ledger revokes A's read fact, moving to revision 1. Before any removal commit, B releases a message a in epoch 0, so $v(a) = 1$ and $e(a) = 0$. Since $A \in M_0$, the group protocol has given A the key, so $\{A\} \vdash k_0$. Yet $A \notin \mathrm{Rd}_1(g)$. □

The proof uses no broken cryptography, no crash and no replay. It sits in the discordance window of Fig. 1. Any concordant protocol must ensure that no read release ever falls in such a window.

IV. THE CONSTRUCTION

The ledger keeps, besides its facts, four pieces of state: a *fence* $\varphi \in \{\mathsf{open}, \mathsf{closed}\}$ for each group; the canonical epoch e^*; a *roster record* $\mathcal{R} = \{(p, J_p)\}$, listing for each member the justification under which it was admitted; and a set of spent (action, receiver) pairs. Every ledger step is atomic. The construction consists of four rules.

Rule F (fence). In the same atomic step that changes the live set from L_v to L_{v+1}, the ledger sets $\varphi \leftarrow \mathsf{closed}$ if some recorded justification is no longer live:

$$\exists (p, J_p) \in \mathcal{R}:\ J_p \not\subseteq L_{v+1} \implies \varphi \leftarrow \mathsf{closed}. \tag{7}$$

Rule C (commit). While $\varphi = \mathsf{closed}$, a member q whose own record is still live builds an MLS commit for epoch e^*+1. The commit removes *exactly* the members whose records are dead and carries a fresh path secret (an UpdatePath). The ledger accepts the commit only if its parent is the current epoch, which is a compare-and-swap. It installs

$$\begin{aligned} \mathcal{R} &\leftarrow \{(p, J_p) \in \mathcal{R} : J_p \subseteq L_v\}, \\ M_{e^*+1} &= \{p : (p, \cdot) \in \mathcal{R}\}, \end{aligned} \tag{8}$$

and opens the fence. Admission is the same step in reverse. With the fence open, a commit may add p together with a J_p that satisfies $J_p \Vdash_v (p, \mathrm{read}, g)$. For each installed commit, the ledger signs an *acknowledgement* binding the exact commit, the new epoch and the current revision v. A client merges a commit only if it holds that acknowledgement. Commits carry only Add and Remove proposals (or none) and the required path. No external commits, group-context changes, resumption pre-shared keys or re-initializations are accepted.

Rule R (release). The ledger releases a read action a only if $\varphi = \mathsf{open}$ and the sender's epoch equals e^*. It releases a write or control action of type α only if the presented justification satisfies $J \Vdash_v \alpha$. In each case it signs a certificate binding the action identifier, a hash of the frozen content, the cut v, the epoch, and J or $\mathcal{R}$. The sender then protects the content once, with the hash of the certificate as MLS associated data.

Rule A (accept). An honest receiver accepts an action only if the certificate verifies and binds the exact bytes received, and only after the ledger has atomically marked the pair (action, receiver) as spent.

Rules F and C govern *who holds keys*, and Rules R and A govern *what is released and accepted*. The roster record makes the ledger deliberately pessimistic. A member whose recorded justification died is removed, even if some other justification would still cover it. That costs availability, never safety.

V. RESULTS

A. The fence lemma

Lemma 1 (Fence)**.** *In every reachable state of the ledger,*

$$\varphi = \mathit{open} \implies \forall (p, J_p) \in \mathcal{R}:\ J_p \subseteq L_v, \tag{9}$$

where v is the current revision. Moreover the parties of $\mathcal{R}$ are exactly M_{e^*}.

Proof. By induction over the ledger's steps. Initially every record was admitted with a live justification and $\varphi = \mathsf{open}$. A step of the ledger is one of the following.

1. A *change of the live set* from L_v to L_{v+1} (a revocation or a new grant). If every record is still live, Eq. (9) continues to hold. Otherwise Rule F closes the fence in the same step, and the implication holds vacuously.

2. A *removal commit* (Rule C). It keeps exactly the live records and opens the fence, so Eq. (9) holds

by construction. The new roster equals the kept records by Eq. (8), and the compare-and-swap ensures that only one child of e^* is installed.

3. An *admission* (Rule C with the fence open). It adds one record whose justification satisfies $J_p \Vdash_v (p, \mathrm{read}, g)$, hence $J_p \subseteq L_v$ by Eq. (2). The revision is unchanged.

4. A *release, acceptance or spend* (Rules R and A). It changes none of v, L_v, $\mathcal{R}$ or φ.

Steps taken locally by clients (processing a commit, crashing, restoring an old snapshot) do not change ledger state. A client that has not installed e^* cannot obtain a release, because Rule R compares its epoch with e^*. □

Lemma 2 (Roster inclusion)**.** *For every released read action a in group g,*

$$M_{e(a)} \subseteq \mathrm{Rd}_{v(a)}(g). \tag{10}$$

Proof. By Rule R, at the release $\varphi = \mathsf{open}$ and $e(a) = e^*$. Take $p \in M_{e(a)}$ with record (p, J_p). Lemma 1 gives $J_p \subseteq L_{v(a)}$. When p was admitted, $J_p \Vdash (p, \mathrm{read}, g)$ held, so $\mathrm{Scope}(J_p, (p, \mathrm{read}, g))$ holds, and Scope does not depend on the revision. By Eq. (2), $J_p \Vdash_{v(a)} (p, \mathrm{read}, g)$, that is, $p \in \mathrm{Rd}_{v(a)}(g)$. □

Lemma 2 is purely combinatorial. It mentions no key and no adversary. It says that the ledger never releases a read action into an epoch whose roster contains a non-reader. What remains is to connect rosters to keys.

B. Key confinement

Lemma 2 concerns rosters. To reach keys we need one property of MLS. Informally, an epoch key leaks only through a member of that epoch. We state it under five conditions, each enforced by the construction or by the choice of parameters.

K1. *Conformance.* Every commit is processed as RFC 9420 prescribes, so a commit containing a Remove carries a fresh UpdatePath. Honest members draw good randomness.

K2. *Member-only, modeled commits.* The ledger installs only commits sent by a current member. A commit carries Add or Remove proposals or none, a path where required, and possibly a new signature key for the committer. There are no external commits, no external self-add proposals, no group-context changes, and no re-initialization or branching.

K3. *One road to message keys.* Message keys are derived from the epoch secret only through the key schedule and the secret tree. No exporter secret is used outside Rule R.

K4. *Strong primitives.* Members and the ledger sign with a strongly unforgeable (SUF-CMA) scheme, and the public-key encryption of the suite is IND-CCA. Both suites of the implementation are discussed after Proposition 3.

K5. *Fresh incarnations.* A party admitted or re-admitted enters with signature, initialization and leaf keys that no other incarnation ever used.

Proposition 2 (Key confinement, symbolic)**.** *Assume K1–K5, in the Dolev–Yao model. Let e be an epoch on the canonical chain and S a set of parties with $S \cap M_e = \emptyset$. An attacker that holds every state of every member of S, at every time, can derive neither the epoch secret k_e nor any message key of epoch e:*

$$S \cap M_e = \emptyset \implies S \not\vdash k_e. \tag{11}$$

Proof. Let p be an honest member of epoch e, for instance the sender of the message in question. Wallez, Protzenko and Bhargavan prove, in a machine-checked DY* development, a confidentiality theorem for TreeKEM from p's point of view [4, 15]. If the attacker knows the epoch secret K_n, then one of ten listed compromise cases holds. Appendix A gives the full list. The cases fall into three groups.

(i) Eight cases require that the attacker compromised a state belonging to a participant or joiner of epoch n, or to the inviter, who is a participant. The state may be an epoch secret, initialization key, signature key or node keys. By K5 the states of an incarnation are disjoint from those of every other incarnation, and $S \cap M_n = \emptyset$, so none of these states is ever compromised.

(ii) One case requires K_{n-1} together with an add-only commit. Then $M_{n-1} \subseteq M_n$, so $S \cap M_{n-1} = \emptyset$, and induction on n applies.

(iii) One case applies the theorem to the inviter's view of the same epoch and roster. The chain of inviters is finite and ends at the creator of the group, whose own case belongs to group (i).

Several cases also require the attacker to know the pre-shared keys. Those conjuncts only add conditions. Hence the attacker does not know k_e. Message keys are derived from k_e by key derivation functions. In the symbolic model a derived value is known only from its inputs or from a state that stores it. The states of members of M_e are not compromised, and K3 excludes other derivations. □

The machine-checked theorem covers commits with any mix of add, remove and update proposals, blank nodes, unmerged leaves, add-only commits, Welcome messages and pre-shared keys. It covers neither external commits nor bad randomness, which is why K1 and K2 appear. The grouping into (i)–(iii) is the manual step

that Ref. [15] itself uses to derive remove-security from its theorem.

Proposition 3 (Key confinement, computational)**.** *Assume K1–K5. By K2, every commit on the chain is an operation modeled by ETK [3]. Admission is a commit with Add proposals, repair a commit with Remove proposals and a path, and rekey a commit with a path and a new signature key. Then every epoch e with $S \cap M_e = \emptyset$ satisfies ETK's predicate* safe*, even when S is fully corrupted, randomness included. Consequently, by Theorem 1 of Ref. [3], the application secret of e is indistinguishable from random, except with the advantage $\mathrm{Adv}^{\mathrm{ETK}}$ of distinguishing ETK from its ideal functionality $\mathcal{F}_{\mathrm{ECGKA}}$. That theorem assumes IND-CCA public-key encryption and SUF-CMA signatures, in the random-oracle model.*

Proof. ETK's predicate declares an epoch unsafe if some member's secrets are exposed in it, or if one of five traversal clauses, a)–e), holds. Appendix A shows that none applies. Exposures are recorded only for members of the epoch, and none of them is in S. Clause a) needs an injected commit, which requires knowledge of the parent epoch. Clauses b) and c) need an added member whose signature key is exposed or who is corrupted later. By K5 and $S \cap M_e = \emptyset$, no such member exists. Clause d) needs some party's state at e to be known. For members of M_e this fails by K1 and K5. For a removed member, ETK's knowledge relation stops at the commit that removed it, whose secrets-replaced flag cuts the propagation. Clause e) concerns external commits, excluded by K2. □

Instantiating K4. The implementation offers two MLS cipher suites. In the classical suite (0x0001) signatures are Ed25519 with strict verification, which is SUF-CMA [16], and HPKE uses DHKEM over X25519, which is IND-CCA under the strong Diffie–Hellman assumption in the random-oracle model [22]. The default post-quantum suite (0x0052) uses ML-DSA-44 and X-Wing. FIPS 204 states SUF-CMA as a design goal of ML-DSA [17]. For Dilithium, on which ML-DSA is based, Ref. [18] bounds the SUF-CMA advantage in the quantum random-oracle model by MLWE, SelfTargetMSIS and MSIS, for the deterministic signing variant that the implementation uses. Reference [19] later found and corrected a gap in the chosen-message step of that proof. We have not checked whether the corrected proof covers the strong case, nor whether the bound carries over unchanged to the modifications FIPS 204 made. X-Wing is IND-CCA if either X25519 (strong Diffie–Hellman, random-oracle model) or ML-KEM-768 is [20]. HPKE in base mode is a KEM–DEM composition, so it is IND-CCA when its KEM is IND-CCA and its AEAD is secure [21]. One limit remains. ETK's Theorem 1 is proved in the classical random-oracle model. With the post-quantum suite, Proposition 3 therefore holds against classical adversaries; a quantum random-oracle version of ETK is open.

Where the policy binding lives. An earlier design wrote the policy revision into the MLS group context through a GroupContextExtensions proposal in each repair commit. MLS would then confirm the revision cryptographically. ETK does not model that proposal type, and none of our proofs needs the binding. Rule R already binds each release to the epoch the ledger acknowledged. We therefore place the binding in the ledger's acknowledgement. The group context keeps at most a constant extension, fixed at creation, that names the ledger's key and the group, and that acts as part of the group's identity. Extending ETK's proof to group-context changes would let the binding return to MLS; we leave that question open. A caveat on scope: we matched the statements of the conference version of Ref. [3], not the proof in its full version.

Why K2 cannot be dropped. ETK observes that a party whose state was compromised can rejoin a group by external commit, signing with the signature key it kept [3]. In our setting that party is precisely a revoked reader. It keeps its old signature key, signs an external commit, and rejoins without the ledger's consent. The finite-state search of Sec. VI finds this attack as soon as the sequencer accepts external commits.

C. Read concordance: the factorization theorem

Theorem 1 (Factorization)**.** *Under K1–K5 the construction is read-concordant in the symbolic model: Eq. (5) holds in every execution. Computationally, under the hypotheses of Proposition 3, an adversary that causes at most q_m read messages has*

$$\mathrm{Adv}^{\mathrm{read}} \;\le\; \mathrm{Adv}^{\mathrm{euf}}_{\mathrm{ledger}} + \mathrm{Adv}^{\mathrm{ETK}} + \epsilon_{\mathrm{RO}} + q_m \,\mathrm{Adv}^{\mathrm{aead}}, \tag{12}$$

where ϵ_{RO} is the random-oracle loss of the secret-tree derivation.

Proof. Symbolic. Fix a released read action a and let C_a be the parties that were not readers at its cut. Lemma 2 gives $M_{e(a)} \subseteq \mathrm{Rd}_{v(a)}(g)$, hence $C_a \cap M_{e(a)} = \emptyset$. Proposition 2 with $S = C_a$ gives $C_a \not\vdash k_{e(a)}$. So any party that derives $k_{e(a)}$ lies outside C_a, which is Eq. (5).

Computational. We use four games.

- G_0 is the real game.
- G_1 aborts if an honest client installs an epoch, or uses a release, that the ledger did not sign. This costs $\mathrm{Adv}^{\mathrm{euf}}_{\mathrm{ledger}}$. In G_1 every honest sender protects only under the canonical epoch of a genuine release, so Lemma 2 gives $C_a \cap M_{e(a)} = \emptyset$ for the challenge.
- G_2 replaces the MLS group-key layer by $\mathcal{F}_{\mathrm{ECGKA}}$. The ledger, the adapters and the adversary together play ETK's environment, at a cost of

$\mathrm{Adv}^{\mathrm{ETK}}$. By Proposition 3 with $S = C_a$, the challenge epoch is safe, so its application secret is uniformly random.

- G_3 replaces the message keys derived from that secret by random keys, at a cost of ϵ_{RO}. Replacing each of at most q_m ciphertexts by an encryption of zeros then costs $\mathrm{Adv}^{\mathrm{aead}}$ per message.

In the last game the bit b is hidden. □

Theorem 1 is where the factorization pays off. All authorization reasoning sits in Lemma 2, and all reasoning about MLS sits in key confinement. Neither knows about the other, and the theorem is their composition.

D. Write concordance

Theorem 2 (Write concordance)**.** *Assume the ledger is atomic and its signing key is secret. Then for writes, admin commits and admissions,*

$$\mathrm{Adv}^{\mathrm{write}} \;\le\; \mathrm{Adv}^{\mathrm{euf}}_{\mathrm{ledger}} + \mathrm{Adv}^{\mathrm{cr}}_{H}, \tag{13}$$

where $\mathrm{Adv}^{\mathrm{write}}$ *is the probability that an honest receiver accepts an action violating Eq.* (6)*.*

Proof. Suppose an honest receiver accepts bytes b under certificate c. By Rule A, c verifies and binds $H(b)$. Either the ledger issued c, or the adversary forged it, which is the first term. If the ledger issued c, it did so in a release step for some action with content hash $H(b')$. If $b \neq b'$ we have a collision, which is the second term. Otherwise the accepted action is the released one, and Rule R checked $J \Vdash_v a$ at the release revision $v = v(a)$. Each receiver accepts a release at most once because the spend mark is set atomically at the ledger before acceptance. That fact is deterministic, given atomicity. □

No property of MLS key secrecy enters Theorem 2. A revoked writer may keep its group signing key and remain a reader. It still cannot obtain a release, so honest receivers discard what it sends. MLS sender authentication adds attribution (the sender of the MLS message equals the actor in the certificate), not authorization.

E. Delegation, invocation and cascading revocation

Delegation needs no new rule. If B holds a root read fact r_B and delegates read to C with fact d_{BC}, then C's justification is $J_C = \{r_B, d_{BC}\}$. Revoking r_B removes it from the live set, and by Eq. (2) every justification containing r_B dies with it. In particular J_C dies, Rule F closes the fence, and Rule C removes C together with B. Cascading revocation is thus a consequence of locality, not an extra mechanism.

An *invocation* i lets an executor E act on behalf of an origin O. Its justification is the union

$$J = J_O \cup J_E \cup \{i\}, \tag{14}$$

and Scope requires that the requested right and resource lie in the scope of all three parts. The effective authority is therefore the *intersection* of the origin's and the executor's. Theorem 2 applies unchanged. Revoking any fact of the origin's chain stops the executor at the next release. The confused-deputy failure, where only $J_E \cup \{i\}$ is checked, is exactly the omission of J_O from Eq. (14). Section VI exhibits it as a counterexample.

F. Progress and the no-go for offline release

Safety alone is cheap, since a ledger that never releases anything is trivially concordant. The construction also makes progress.

Proposition 4 (Progress)**.** *Suppose that after some revision* v^* *there are no further policy changes. Suppose also that the ledger is reachable, that at least one member with a live record keeps acting, that messages between it, the ledger and the other members are eventually delivered, and that the repair commit is processed successfully by honest members. Then the fence is eventually open, and every later release requested by a member with a live record succeeds.*

Proof. If the fence is closed after v^*, some record is dead and, by assumption, some member q has a live record, so Rule C is enabled for q. Delivery and successful processing let the ledger install the commit, which opens the fence. With no further policy change, Rule F never fires again, so the fence stays open. Rule R's conditions then hold for every member with a live record at e^*. □

The processing assumption matters. The sequencer sees only the public part of a commit and cannot verify its confirmation tag. A commit that passes public checks but fails to process would win the compare-and-swap and strand the group. That outcome is safe but not live. Requiring a second continuing member to confirm the new epoch before the fence opens removes this case, and it is how the implementation proceeds (Sec. VII).

Proposition 5 (No-go for offline release)**.** *A protocol that lets a sender emit a fresh read-protected message without contacting the ledger cannot guarantee concordance at the release cut of Eq.* (4)*.*

Proof. Consider two executions that the disconnected sender cannot distinguish. In the first nothing happens at the ledger. In the second a reader $r \in M_e$ is revoked. If the sender emits under k_e in the first execution, it emits identically in the second. There r holds k_e but is not a reader at any cut later than the revocation, so Eq. (5) fails. If it does not emit in the first, the protocol is unavailable. □

Proposition 5 is elementary, in the spirit of the CAP argument [14]. Its content is the identification of what must be given up. Offline emission is possible only under a weaker cut, for instance a lease whose issue revision stands in for the cut until the lease expires.

VI. FINITE-STATE EVIDENCE

The proofs above are by hand. As an independent check we built an executable model of the construction (`formal/model/concordance_model.py`) and enumerated every reachable state within fixed resource bounds. The model has three parties. A and B hold root read facts, and C reads through a delegation d_{BC} from B. For writing, A and C hold root write facts, and an invocation lets C write for A. The bounds are three policy changes, one re-issue per fact, three epochs, two read releases, two write releases and three acceptances. These bounds make the state space finite, so the search is exhaustive within them rather than cut off at a depth.

The adversary is implicit and maximal. Every party keeps every epoch key it ever obtained, and every party is treated as possibly corrupt. No transition consults the property being checked. The check of Eq. (5) is literally "for each read release and each party that holds its epoch key, was that party a reader at the release?" The model takes key distribution by roster as the meaning of a commit, so key confinement is built into the good model. Conditions K1 and K2 are tested separately: one mutant drops the fresh path secret, another lets the sequencer accept an external commit.

The good model has 1 556 555 reachable states and 4 104 565 transitions. Of these states, 1 457 995 contain a read release and 1 503 996 contain an accepted write, so the properties are exercised, not vacuous. No state violates read concordance, write concordance, the fence lemma or key confinement. In every reachable state with a closed fence, a member with a live record, and epochs to spare, a repair is enabled. The search also reaches, by shortest paths, a read release after a repair, the repair of a cascaded revocation, the re-admission of a revoked party under a new generation, and an accepted write by invocation.

Each of nine *mutants* deletes exactly one rule or condition. For each, the same search finds a shortest counterexample without guidance (Table I). Every rule, and condition K2, is therefore load-bearing in the model. The last row is the rejoin attack of Ref. [3], transposed to a revoked reader.

The model is bounded and symbolic. It does not prove the theorems for arbitrary group sizes, and it contains no cryptography. Its value is complementary to the proofs: it shows that the rules are individually necessary and that the definitions are not vacuous.

TABLE I. Each mutant deletes one rule or condition. The search finds the shortest trace violating read (R) or write (W) concordance.

Deleted rule	Fails	Shortest counterexample
F: fence	R	revoke r_A; release in e=0
F: cascade	R	revoke r_B; repair keeps C; release
C: path secret	R	revoke r_A; repair; release in e=1
C: new epoch	R	revoke r_A; drop A silently; release
R: epoch check	R	revoke r_A; repair; release in e=0
R: origin chain	W	revoke w_A; C writes for A; accept
A: one-shot	W	release; accept; accept again
A: certificate	W	accept a write never released
K2: members only	R	revoke r_A; repair; A rejoins; release

TABLE II. Mathematical objects and their realization.

Model	Implementation
Ledger, L_v, revision	kernel, `policy_revision`
Justification, Scope	grants, delegation chains, R-8
Invocation, Eq. (14)	invocation records
Rule F	`revoke` with descendant cascade
Rule C	`repair_stage`, `repair_open`
Rule R	`release` and release certificate
Rule A	adapter receive path, `consume`
K1–K5	Table V, Appendix A

VII. CORRESPONDENCE WITH THE IMPLEMENTATION

The construction is being implemented in Rust on OpenMLS, in the `agent-trust` repository. Table II lists where each element lives. The implementation's single-writer SQLite kernel realizes the atomic ledger. It is not replicated, which is a deployment choice rather than a model assumption.

Four differences between the code and the model are worth stating.

- The implementation is *stricter* in three places. First, a receiver delivers only while the epoch and generations are unchanged, whereas the model allows a pre-cut message to be accepted later. Theorems 1 and 2 hold a fortiori, since only acceptances are removed, but Proposition 4 then promises nothing for in-flight messages. Second, the central fence opens only after a second continuing member confirms the repaired epoch, which discharges the processing assumption of Proposition 4. Third, every application message is both a write by its sender and a read-protected action for the group. Its release therefore checks both conditions of Rule R, and it also checks directly that every member of the current roster holds a

live read justification, which is the conclusion of Lemma 2.

- Delegations carry wall-clock validity windows, evaluated at release, so their expiry is not a ledger event and Eq. (2) is not met literally. The theory predicts the consequence. If a member's delegation expires, releases are refused, which is safe. But the fence never closes, so no repair can remove that member, which is a loss of progress. Recording expiry as a ledger revocation restores Eq. (2).

- Condition K1 asks for good randomness. The adapter draws path secrets directly from the selected entropy source. The HPKE encapsulation inside OpenMLS draws from a deterministic random bit generator seeded from that same source. Both are good randomness, so K1 holds. The distinction matters only for claims about where the entropy comes from.

- The repair commit currently still carries a GroupContextExtensions proposal with the policy binding. This is the earlier design discussed in Sec. V B, and it lies outside ETK's model. A proposed correction moves the binding into the ledger's acknowledgement and freezes the context extension. Until it is applied, Proposition 3 covers the construction but not this one commit type of the code. Proposition 2 covers both.

The implementation's tests (a retained-key adversary on successor traffic, crash barriers, and an independent trace oracle) are evidence for the executions they run. They are not proofs, and the paper does not use them as such.

VIII. RELATED WORK

Group key agreement and MLS. RFC 9420 specifies MLS [1], and RFC 9750 places access control in the application [2]. Continuous group key agreement was formalized with active security by Alwen *et al.* [7], and the modular analysis of MLS-era messaging [6] is the closest proof architecture. ETK analyses the final RFC including external operations [3]. ETK also identifies the rejoin-by-external-commit weakness that condition K2 rules out. TreeSync [5] and the TreeKEM analysis [4] are machine-checked symbolic results. Key confinement rests on Refs. [4] and [3], and Ed25519's strong unforgeability under strict verification on Ref. [16]. None of these works relates an external, independently evolving ledger to the group's epochs.

Administration and policy in groups. Balbás, Collins and Vaudenay define administrated CGKA, in which only admins can change membership [8]. That is the closest cryptographic treatment of our admin right, with policy inside the group rather than in an external ledger. The IETF MIMI drafts give a hub provider the role of ordering messages and enforcing room policy on MLS groups [9], which is architecturally close to our ledger and sequencer, without a concordance theorem. Ink & Switch's Keyhive pairs delegation-chain capabilities for writing with a TreeKEM-like key agreement for reading [10], the same division of labor as here, in a local-first setting. Signal's private group system enforces group access at the server with anonymous credentials [11].

Revocation. Security-mediated RSA withholds per-operation help after revocation while the client keeps its key share [12]. It is the direct precedent for mediating every use, which we do not claim as new. The reference-monitor principle of complete mediation [13] is the engineering ancestor of Rules R and A.

What is new here. We contribute the definition of may–can concordance with an explicit cut, the factorization of read concordance into a combinatorial roster lemma and the key confinement of MLS, derived from published theorems, the uniform treatment of delegation and invocation through locality, and a construction whose every rule is shown necessary.

IX. DISCUSSION

Concordance asks one thing: whoever can exercise a capability on an action may do so at the instant the action was released. Once the cut is fixed, the problem splits cleanly. The ledger keeps the roster honest (Lemma 2), MLS keeps keys inside the roster (Propositions 2 and 3), and signatures keep acceptances honest (Theorem 2).

Four steps remain open, in decreasing order of importance.

1. *Align the code and check the full ETK proof.* Apply the proposed correction that moves the policy binding into the acknowledgement. Then check the proof in the full version of Ref. [3] against the static coalition corruption pattern used here. For the post-quantum suite, check that the corrected ML-DSA proof [19] covers strong unforgeability; a quantum random-oracle version of ETK would extend Proposition 3 to quantum adversaries. Separately, an extension of ETK to group-context changes is a natural question for its authors.

2. *Mechanize Lemma 1 and Theorem 2.* Both are inductive invariants of a small transition system and suit a proof assistant or TLA$^+$ with its proof system.

3. *Other cuts.* Causal and lease cuts trade the ledger contact of Proposition 5 for weaker guarantees. Each needs its own analogue of Lemma 2.

4. *Finer read domains.* Here one group is one read domain. Per-resource read access within a group cannot be expressed by group keys alone and needs per-resource keys or separate groups.

Appendix A: Key confinement against the published theorems

This appendix records the case-by-case match behind Propositions 2 and 3. Throughout, S is a set of parties with $S \cap M_e = \emptyset$, and the attacker holds every state of every member of S.

1. The symbolic theorem

The TreeKEM confidentiality theorem [15, Sec. 6.4.3] is stated from the view of an honest participant p of epoch n, with participants $p_i^{(n)}$, joiners $j_i^{(n)}$ and inviter p_{inv}. If the attacker knows K_n, one of the cases in Table III holds. Most cases carry an extra conjunct, "attacker knows the epoch's pre-shared keys"; it only adds conditions, and we omit it. Group (i) is excluded by $S \cap M_n = \emptyset$ and K5. Groups (ii) and (iii) reduce to the same statement for an earlier epoch or for the inviter, and induction closes them.

TABLE III. The ten cases of the TreeKEM theorem and why none applies.

Case	Attacker has compromised	Group
1	the epoch secret held by a participant	(i)
2	a joiner's initialization key	(i)
3	a joiner's signature key, before it was checked	(i)
4	K_{n-1}, and the commit is add-only	(ii)
5	K_{n-1} and a participant's current node keys	(i)
6	K_{n-1} and a participant's signature key, before it was checked	(i)
7	K_{n-1} and a stale participant's initialization key	(i)
8	the inviter's signature key, before it was checked	(i)
9	(the inviter's own view of epoch n)	(iii)
10	the creator's epoch-0 secret	(i)

The model behind this theorem has an untrusted delivery service. Each participant processes the commits it receives in order; in our construction the ledger fixes that order. It has symbolic cryptography and honest randomness, and it has no external commits. The last two facts are why conditions K1 and K2 are needed.

2. The computational theorem

ETK's Theorem 1 [3] realizes $\mathcal{F}_{\text{ECGKA}}$, in which the application secret of an epoch c is random whenever $\mathsf{safe}(c)$ holds. Here

$$\mathsf{safe}(c) \iff \neg\big[(*, \mathsf{true}) \in \mathsf{exp}(c) \ \vee \ \mathsf{can\text{-}traverse}(c)\big].$$

Table IV lists each disjunct and why it is false.

TABLE IV. ETK's unsafe conditions for an epoch c, and why each is false when $S \cap M_c = \emptyset$ and K1–K5 hold.

Condition	Why it is false
exposure in c	only members of c are exposed in c; none is in S
a) injected root with exposed key	every canonical commit has a parent; injecting needs the parent epoch's secrets
b) member added with exposed key	added members are in M_c; K5 gives fresh keys
c) Welcome opened by a later corruption	added members are in M_c and never corrupted
d) some party's state at c known	members: never exposed, good randomness (K1); removed parties: the removing commit replaces their secrets and cuts propagation
e) external commit	excluded by K2

ETK also lets an exposed signature key inject external proposals and commits at any time. K2 makes the sequencer reject them. A regular commit can be injected only when the parent epoch's secrets are known, which fails by induction along the chain. ETK's operations are regular commits with add, remove, update and pre-shared-key proposals, external self-add proposals, and external commits. Under K2 the construction issues three kinds of commit. Admission adds members. Repair removes members, with the committer's own Remove proposals (ETK's regular remove) and a path. Rekey carries no proposals, a path and a new signature key, which is ETK's commit with an updated signature key. The constant context extension fixed at creation adds no operation.

3. Status in the implementation

Table V lists where the implementation enforces each condition. Every entry was checked in the source tree, including the pinned OpenMLS fork, whose commit processing rejects a commit that needs a path but lacks one (check ValSem201).

TABLE V. How the implementation meets K1–K5.

	Where it is enforced
K1	repair commit built with `force_self_update(true)`; OpenMLS rejects a remove without a path
K2	sequencer accepts only `Sender::Member` commits (`process_commit`)
K3	no exporter secret is used in `src/`
K4	default suite `0x0052`: ML-DSA-44 signatures, X-Wing HPKE; classical suite `0x0001`: Ed25519 verified with `verify_strict`, DHKEM(X25519). OpenMLS and the ledger use the same signature scheme
K5	re-admission by a new KeyPackage; other rights by an update with a new signer
pending	the repair commit still calls `propose_group_context_extensions`; the proposed correction removes it

[1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420 (IETF, 2023).

[2] B. Beurdouche, E. Omara, E. Rescorla, K. Cohn-Gordon, and R. Barnes, *The Messaging Layer Security (MLS) Architecture*, RFC 9750 (IETF, 2025).

[3] C. Cremers, E. Günsay, V. Wesselkamp, and M. Zhao, ETK: External-operations TreeKEM and the security of MLS in RFC 9420, in *Eurocrypt 2026*; IACR ePrint 2025/229.

[4] T. Wallez, J. Protzenko, and K. Bhargavan, TreeKEM: A modular machine-checked symbolic security analysis of group key agreement in Messaging Layer Security, in *IEEE Symposium on Security and Privacy* (2025), pp. 4375–4390; IACR ePrint 2025/410.

[5] T. Wallez, J. Protzenko, B. Beurdouche, and K. Bhargavan, TreeSync: Authenticated group management for Messaging Layer Security, in *USENIX Security* (2023).

[6] J. Alwen, S. Coretti, Y. Dodis, and Y. Tselekounis, Modular design of secure group messaging protocols and the security of MLS, in *ACM CCS* (2021).

[7] J. Alwen, S. Coretti, D. Jost, and M. Mularczyk, Continuous group key agreement with active security, in *TCC* (2020).

[8] D. Balbás, D. Collins, and S. Vaudenay, Cryptographic administration for secure group messaging, in *USENIX Security* (2023); IACR ePrint 2022/1411.

[9] R. Barnes *et al.*, *The MIMI+MLS Protocol*, draft-ietf-mimi-protocol-06 (IETF, 2026), work in progress; R. Mahy, *Room Policy for the MIMI Protocol*, draft-ietf-mimi-room-policy-04 (IETF, 2026), work in progress.

[10] Ink & Switch, *Keyhive lab notebook* (2024–2025), https://www.inkandswitch.com/keyhive/notebook/.

[11] M. Chase, T. Perrin, and G. Zaverucha, The Signal private group system and anonymous credentials supporting efficient verifiable encryption, in *ACM CCS* (2020).

[12] D. Boneh, X. Ding, G. Tsudik, and C. M. Wong, A method for fast revocation of public key certificates and security capabilities, in *USENIX Security* (2001).

[13] J. P. Anderson, *Computer Security Technology Planning Study*, Tech. Rep. ESD-TR-73-51 (U.S. Air Force, 1972).

[14] S. Gilbert and N. Lynch, Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services, SIGACT News **33**, 51 (2002).

[15] T. Wallez, *Formal Verification for High Assurance Messaging Protocols*, Ph.D. thesis, Inria Paris (2025), Ch. 6, https://www.twal.org/pdf/thesis.pdf.

[16] J. Brendel, C. Cremers, D. Jackson, and M. Zhao, The provable security of Ed25519: Theory and practice, in *IEEE Symposium on Security and Privacy* (2021).

[17] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204 (2024), Sec. 3.1.

[18] E. Kiltz, V. Lyubashevsky, and C. Schaffner, A concrete treatment of Fiat–Shamir signatures in the quantum random-oracle model, in *Eurocrypt 2018*; the bound for Dilithium is restated in L. Ducas *et al.*, *CRYSTALS-Dilithium Algorithm Specifications and Supporting Documentation*, version 3.1 (2021), Sec. 6.2.

[19] M. Barbosa, G. Barthe, C. Doczkal, J. Don, S. Fehr, B. Grégoire, Y.-H. Huang, A. Hülsing, Y. Lee, and X. Wu, Fixing and mechanizing the security proof of Fiat–Shamir with aborts and Dilithium, in *Crypto 2023*; IACR ePrint 2023/246.

[20] M. Barbosa, D. Connolly, J. D. Duarte, A. Kaiser, P. Schwabe, K. Varner, and B. Westerbaan, X-Wing: The hybrid KEM you've been looking for, IACR Communications in Cryptology **1**(1) (2024); IACR ePrint 2024/039.

[21] R. Cramer and V. Shoup, Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack, SIAM J. Comput. **33**, 167 (2003).

[22] M. Abdalla, M. Bellare, and P. Rogaway, The oracle Diffie–Hellman assumptions and an analysis of DHIES, in *CT-RSA 2001*.